MILO

IMPLEMENTATION PLAN

The complete, ordered engineering execution roadmap — from an empty repository to a tested, signed, installable Windows release.

Version 1.0 | 02 November 2026 | Windows-first desktop application

Purpose

This document tells Codex what to build, in what order, how to verify each layer, when to stop and repair a foundation, and what constitutes a genuinely finished MILO release. It is an execution plan, not a replacement for the Product Requirements Document, Technical Requirements Document, or UI/UX Design Debrief.

Build principle

Build one dependable vertical slice at a time. Every phase must compile, run, be tested, and leave the application in a usable state before the next phase begins. Do not implement all screens first and wire them later.

Document control and how Codex must use this plan

This plan is the authoritative sequence of implementation work. The companion MILO documents define what the product is and how it should look; this plan defines the order in which the engineering work is executed.

Document	Authority
Product Requirements Document (PRD)	Product intent, user journeys, scope, priorities, acceptance outcomes.
Technical Requirements Document (TRD)	Architecture, platform integration requirements, data contracts, security and performance constraints.
UI/UX Design Debrief	Visual system, layout, components, motion, states, responsive behaviour and interaction design.
This Implementation Plan	Build order, phase gates, integration sequence, verification, release and handover.

Mandatory operating rules for Codex

- **Read before coding** — Inspect all three companion documents and this plan in full. Create a concise internal implementation checklist from them. Do not infer missing requirements from screenshots or invent replacement behaviour.
- **Work in sequence** — Complete phases in order. A later phase may be researched early, but its production implementation must not bypass the dependency gates.
- **Inspect the actual repository first** — If code already exists, audit it and map it to these phases. Preserve sound work; do not blindly overwrite or regenerate the project.
- **No fake functionality** — Never make a control appear operational if it is only a mock. Label prototype data clearly and remove mocks before release.
- **Small, reviewable increments** — Implement a coherent vertical slice, run checks, inspect the result, then continue. Avoid enormous one-shot rewrites.
- **Windows is the release target** — Treat Windows 10/11 compatibility, WebView2 availability, DPI, multi-monitor behaviour, sleep/resume, and installer behaviour as first-class requirements.
- **Native capabilities need proof** — For every OS integration, build a minimal technical spike and verify it on Windows before designing a large abstraction around it.
- **Keep the app runnable** — After every phase, `dev` launch and production build must remain healthy unless the phase explicitly concerns packaging and documents why.
- **Security by default** — Use least privilege, explicit Tauri capabilities, strict command validation, safe file handling, and no unnecessary shell execution.
- **Document decisions** — Record chosen crates, Windows APIs, limitations, known issues, and reasons in architecture decision records (ADRs).
- **Never silently destroy user data** — File staging is non-destructive. No silent move, rename, overwrite, delete, or upload.
- **Stop when a gate fails** — Fix the failed phase before moving forward. Do not stack new features on top of broken foundations.

Definition of done for every phase

- Implementation exists in the real application, not only in a design mock or isolated demo.
- Relevant automated tests pass and any manual Windows checks are recorded.
- No new compiler errors, TypeScript errors, lint errors, or unexplained warnings are introduced.

- Loading, empty, error, disabled, permission-denied, and recovery states are considered where applicable.
- The feature survives application restart if its data is intended to persist.
- Accessibility, keyboard interaction, reduced motion, and scaling are checked where relevant.
- Documentation, settings, telemetry policy, and user-facing copy are updated where needed.
- A concise phase report states files changed, tests run, results, known limitations, and the next approved phase.

Executive roadmap

Stage	Phases	Outcome
Foundation	0–4	Repository, toolchain, architecture, shell, design system and persistence are reliable.
Core product	5–8	Floating island, navigation, system deck and productivity foundation work end-to-end.
Distinctive features	9–12	Tasks/reminders, file staging, media/lyrics and avatar are integrated.
Desktop polish	13–15	Notifications, startup, settings, accessibility, resilience and performance are production-ready.
Release	16–19	Full QA, security review, installer, signing, update path, pilot and handover are complete.

Critical-path warning: the floating window behaviour, Windows native integration layer, and file drag-out workflow are the highest-risk engineering areas. Prove them early with small spikes; do not leave them until the end.

Phase 00 — Pre-flight: understand the product and inspect the machine

Objective: Establish an accurate picture of the existing workspace, developer environment, source documents, and Windows target before changing anything.

Exact execution sequence

1. Locate the current project directory and inspect its full tree. Identify existing code, assets, package files, build scripts, tests, generated files, and any unfinished implementation.
2. Read the PRD, TRD, UI/UX Design Debrief, and this implementation plan. Extract requirements into a traceability checklist with unique IDs. Preserve original requirement wording where useful.
3. Determine the available host OS and target build environment. If Codex is not running on Windows, identify which Windows-only behaviours require a real Windows machine or CI runner.
4. Record installed versions of Node.js, package manager, Rust toolchain, Cargo, Tauri CLI, Windows SDK/build tools, and WebView2 runtime. Do not upgrade blindly; check compatibility first.
5. Confirm the intended application identity: product name MILO, bundle identifier, executable name, publisher placeholder, app-data location, icon assets, and versioning convention.
6. Create a risk register. At minimum include top-edge window positioning, transparent/rounded window rendering, focus behaviour, display/DPI changes, Windows system APIs, external drag-out, media session coverage, lyrics rights/source, and installer/update signing.
7. Identify all external dependencies and licensing obligations. Flag any library with unclear maintenance, licence, or Windows support.
8. Create a baseline report: what exists, what builds, what runs, what is broken, and what is unknown. Do not describe an unverified capability as complete.

Deliverables

- Repository audit report and baseline build log.
- Requirement traceability checklist covering every PRD/TRD/UI requirement.
- Initial risk register and list of decisions requiring a spike.
- Confirmed toolchain and target Windows test environment.

Verification

- Do not redesign the product or simplify scope during pre-flight.
- Do not start feature coding before the repository and requirements audit.
- Do not assume the current environment can validate Windows-native behaviour.

EXIT GATE — Codex can explain the current state of the repository, list the requirements, identify platform constraints, and reproduce a clean baseline build without modifying unrelated files.

Phase 01 — Create the repository structure and engineering workflow

Objective: Create a maintainable project skeleton with repeatable local commands and automated quality checks.

Exact execution sequence

1. Choose and document the package manager and lockfile policy. Use one package manager consistently; commit the lockfile.
2. Create or validate the Tauri v2 + Rust + React + TypeScript project. Keep frontend and Rust backend boundaries explicit.
3. Establish a clear source layout: frontend app, feature modules, shared UI, state, domain models, IPC client, Rust commands, platform services, persistence, migrations, tests, assets, and documentation.
4. Add formatting, linting, type-checking, Rust formatting, Rust linting, and test commands. Pin or document tool versions.
5. Add `.editorconfig`, `.gitignore`, contribution notes, environment example files, and a concise README with setup/build/test commands.
6. Set up Git branches or a trunk-based workflow, conventional commit guidance if desired, and pull-request checklists.
7. Create CI that installs dependencies, checks formatting, runs lint/type checks, runs Rust tests, builds the frontend, and performs a Tauri build on a Windows runner.
8. Add dependency update and vulnerability scanning workflows appropriate to the project. Avoid auto-merging security-sensitive updates.
9. Define versioning and release channels: development, preview/beta, and stable. Keep release signing secrets out of the repository.

Deliverables

- Compiling starter application.
- Consistent source tree and naming conventions.
- Local developer commands and CI workflow.
- README and contribution guidance.
- Dependency and release hygiene.

Verification

- Do not add every Tauri plugin immediately. Add a dependency only when its phase requires it.
- Do not place all frontend logic in one App component or all backend logic in one Rust command module.

EXIT GATE — Fresh clone → install → lint → type-check → tests → build succeeds using documented commands; CI repeats the checks successfully on Windows.

Phase 02 — Prove the Windows desktop shell and native window behaviour

Objective: Resolve the most fundamental product risk: MILO must behave like a small floating desktop island attached to the top-centre screen edge, not like a conventional maximised application.

Exact execution sequence

1. Create the smallest Tauri window that launches on Windows and renders a diagnostic surface.
2. Verify window dimensions, position, decorations, shadow, transparency, always-on-top behaviour, taskbar visibility, focus, and z-order. Record which properties are native and which are simulated.
3. Implement a window controller in Rust with a narrow interface: initialise, position, resize, show, hide, focus, blur handling, monitor query, and shutdown.
4. Calculate top-centre positioning against the active target monitor's work area and physical bounds. Define how the app chooses a monitor when multiple displays exist.

5. Test Windows scaling factors (100%, 125%, 150%, 200%), mixed-DPI monitors, display resolution changes, monitor unplug/reconnect, orientation changes, and taskbar placement.
6. Test shell states: collapsed, expanding, expanded, collapsing, hidden, and temporarily pinned. Use a single state machine rather than independent booleans.
7. Verify that pointer entry, pointer exit, clicks, keyboard focus, and interaction with controls do not cause accidental collapse.
8. Test alt-tab, Win+D, virtual desktops, full-screen applications, remote desktop sessions, lock/unlock, sleep/resume, and Explorer restart.
9. Create a native diagnostic page or log mode that reports monitor bounds, DPI, window bounds, focus state, and transitions.
10. Write ADRs for any platform-specific workaround. If a behaviour cannot be delivered reliably with Tauri's standard window APIs, prototype the smallest Rust/Win32 extension and isolate it.

Deliverables

- A reliable top-edge floating window.
- Window controller and documented state machine.
- Multi-monitor/DPI test matrix.
- Recorded proof of focus and z-order behaviour.
- ADRs for native window workarounds.

Verification

- Do not build the full visual interface yet.
- Do not assume CSS positioning can control the native window's screen placement.
- Do not implement auto-hide solely with a CSS hover selector; window-level pointer and focus behaviour must be coordinated with the native controller.

EXIT GATE — On a real Windows machine, MILO attaches to the intended monitor edge, remains correctly positioned through display changes, expands and collapses without clipping or stealing focus unexpectedly, and does not become an ordinary taskbar window unless that is an intentional product decision.

Phase 03 — Build the frontend foundation and design-token system

Objective: Create the visual and interaction foundations before implementing individual product sections.

Exact execution sequence

1. Install and configure the chosen React/TypeScript build stack and verify strict TypeScript mode.
2. Create the global theme and token files from the UI/UX Design Debrief: colours, gradients, typography, spacing, radii, borders, shadows, blur/material values, elevation, motion durations, easing curves, and z-index layers.
3. Implement the app reset, font loading/fallbacks, icon strategy, focus rings, selection styling, scrollbars, and reduced-motion behaviour.
4. Build primitives: text styles, icon button, standard button, toggle, slider, tooltip, popover, card, badge, divider, input, textarea, list row, empty state, loading state, error state, and confirmation dialog.
5. Implement theme variables and component variants; do not hardcode one-off colours or spacing throughout feature files.
6. Build the visual shell in a static preview route or story gallery so components can be reviewed independently.

7 Implement responsive layout constraints for the island's narrow and expanded widths, scrollable content regions, and safe internal padding.

8 Ensure keyboard focus order, accessible names, tooltip alternatives, contrast, and minimum target sizes are defined at the primitive level.

9 Add visual regression snapshots for the most important primitives and shell states if the test stack supports them.

Deliverables

- Central token library.
- Reusable, accessible component primitives.
- Theme and motion foundation.
- Component preview/gallery.
- Visual QA baseline.

Verification

- Do not build every feature screen with placeholder styling.
- Do not add multiple competing component libraries without a clear reason.
- Do not add decorative animations that obscure basic interaction.

EXIT GATE — Design primitives match the approved design direction, use shared tokens, render correctly in the target WebView2 environment, and pass keyboard/contrast/reduced-motion checks.

Phase 04 — Define domain models, IPC contracts, persistence and event conventions

Objective: Create stable contracts between React, Rust, Windows services, and local storage before feature logic spreads across the application.

Exact execution sequence

1 Define domain entities and TypeScript types for settings, deck state, reminders, tasks, timers, alarms, staged files, media metadata, lyrics metadata, avatar state, notifications, and audit-safe activity events.

2 Define matching Rust structs/enums with explicit serialisation rules. Add schema/version fields where long-lived data may evolve.

3 Define IPC naming conventions and response envelopes. Commands should return typed success data or structured errors; do not leak raw OS errors to the UI.

4 Create a single typed frontend IPC client. Feature modules must call this client rather than invoking arbitrary command strings throughout components.

5 Design event names and payload schemas for system status changes, media changes, timers, reminders, window state, and file staging. Define subscription cleanup and reconnection behaviour.

6 Choose the local database strategy (SQLite is the default recommendation for relational user data) and implement connection lifecycle, migrations, transaction helpers, and backup/export hooks.

7 Keep ephemeral UI state in frontend state; keep durable user data in the database; keep machine capability state in native services. Document each ownership boundary.

8 Implement structured logging with levels, correlation IDs where useful, redaction rules, and user-accessible diagnostic export. Never log file contents, lyrics content unnecessarily, credentials, or sensitive paths by default.

9 Create an error taxonomy: validation, permission, unsupported, unavailable, timeout, conflict, persistence, and internal errors.

10 Write contract tests that verify serialisation, validation, error mapping, and event payload shape.

Deliverables

- Domain model package and Rust counterparts.
- Typed IPC client and command conventions.
- Database connection and migration framework.
- Event contract catalogue.
- Structured logging and error taxonomy.

Verification

- Do not implement business features inside UI components.
- Do not expose unrestricted filesystem or shell commands to the frontend.
- Do not treat every error as a generic string.

EXIT GATE — Frontend and backend can exchange typed data; schema migrations run on startup; contract tests pass; and feature modules have a documented data ownership model.

Phase 05 — Implement application lifecycle, tray, single-instance and startup behaviour

Objective: Make MILO behave like a resident desktop utility with predictable launch, background, restore, and exit semantics.

Exact execution sequence

1 Implement single-instance protection and define what happens when a second launch occurs: focus or reveal the existing island rather than opening duplicates.

2 Create a system tray icon and menu with Show/Hide MILO, Settings, Diagnostics/Logs, and Exit. Ensure menu labels and actions are accessible.

3 Define startup sequence: initialise logging, acquire single-instance lock, load settings, initialise database, register native services, determine monitor, create window, restore state, and show the collapsed island.

4 Define clean shutdown: stop timers and subscriptions, flush durable writes, release native resources, and close background workers.

5 Implement startup-with-Windows as an explicit user-controlled setting. Do not silently enable it during installation.

6 Test launch after Windows sign-in, manual launch, repeated launch, crash/relaunch, Explorer restart, sleep/resume, and clean exit.

7 Add crash-safe recovery for partially completed operations and interrupted database writes.

8 Implement a first-run state marker and versioned onboarding flag, but defer the full onboarding experience to the settings/onboarding phase.

Deliverables

- Single-instance lifecycle.
- System tray and menu.
- Startup/shutdown orchestration.
- Optional Windows startup integration.
- Lifecycle diagnostics.

Verification

- Do not hide all ways to quit the application.
- Do not enable startup automatically without consent.
- Do not persist transient expanded state unless product requirements explicitly call for it.

EXIT GATE — Repeated launches do not create duplicate processes/windows; tray actions work; shutdown is clean; user startup preference is respected; and restart restores the expected collapsed state.

Phase 06 — Implement the island shell and complete navigation

Objective: Deliver the recognisable MILO interaction model with real shell states and navigable sections.

Exact execution sequence

1. Build the collapsed island layout using approved geometry, top-edge attachment, silhouette, lighting, and compact information hierarchy.
2. Implement the expanded shell with header, avatar placement, navigation rail or section selector, content viewport, and footer/quick actions as specified in the design debrief.
3. Connect the frontend shell to the native window controller. A single state machine must coordinate visual animation and actual native window dimensions.
4. Implement open/close triggers: hover, click, keyboard shortcut if configured, tray action, and explicit pin/unpin. Respect focus and pointer capture.
5. Implement animation choreography: reveal, expand, content entrance, section transition, collapse. Avoid animating the entire page tree when only a panel needs to move.
6. Add scroll containment and overscroll behaviour. Verify the cursor can reach all controls without the window disappearing.
7. Implement navigation routing/state and restore the last selected section only if the PRD calls for it. Make the Overview entry point predictable.
8. Add keyboard navigation, Escape behaviour, focus restoration, screen-reader labels, and reduced-motion transitions.
9. Add a temporary debug overlay to show native bounds and UI shell state; keep it developer-only.

Deliverables

- Collapsed and expanded shell.
- Reliable native/UI state synchronisation.
- Working navigation and scroll behaviour.
- Keyboard and accessibility support.
- Shell transition tests.

Verification

- Do not build the individual feature modules as one giant page.
- Do not let animation state become the source of truth for native window geometry.

EXIT GATE — Users can reveal, navigate, interact with, pin, and dismiss the island reliably across the supported display configurations. No feature screen is required to be complete yet, but navigation must be structurally real.

Phase 07 — Build the Overview and System Deck

Objective: Deliver the first meaningful at-a-glance experience and establish a reusable native system-service pattern.

Exact execution sequence

1. Implement the time/date display using local system time and locale-aware formatting. Handle time-zone changes and daylight-saving transitions.
2. Implement battery and power status using a Windows-native service. Model unavailable battery, desktop/no-battery devices, charging, discharging, and critical states.
3. Implement volume read and set operations through a Windows audio integration. Confirm the chosen API can control the intended default endpoint and define behaviour for multiple devices.
4. Implement brightness only where supported by the device/monitor. Detect unsupported external displays and report a clear unavailable state rather than presenting a broken slider.
5. Implement Wi-Fi/network status and Bluetooth status as read-only capability/status first. Do not imply that all radios can be toggled reliably on every Windows device.
6. Implement energy saver/power mode status and controls only after the correct supported Windows API and permission model are verified.
7. Implement refresh/event updates with appropriate polling or native event subscriptions. Avoid high-frequency polling when event-driven updates are available.
8. Build each system tile with loading, supported, unsupported, permission-denied, stale-data, and error states.
9. Implement optimistic UI only for operations with a safe rollback strategy. Reconcile UI state with the native system result.
10. Add system-service diagnostics and device compatibility logging.

Deliverables

- Working Overview/System Deck tiles for supported capabilities.
- Native system service interfaces.
- Capability detection and graceful fallbacks.
- System integration tests on representative hardware.

Verification

- Do not claim universal brightness or radio-toggle support.
- Do not build fake toggles that only change their local UI state.
- Do not use aggressive polling that drains battery or increases idle CPU.

EXIT GATE — Every displayed system control reflects real Windows state. Supported actions change the OS state and confirm the result; unsupported hardware is handled honestly and gracefully.

Phase 08 — Implement productivity foundation: tasks and to-do lists

Objective: Introduce the first persistent productivity domain and prove database-backed CRUD, sorting, and recovery patterns.

Exact execution sequence

1. Implement task/list schema and migration: stable IDs, title, notes, completion, due date, priority, sort order, created/updated timestamps, and archive/delete semantics.
2. Define whether tasks can exist without a list and how the default list behaves. Keep the initial interaction simple.
3. Implement Rust/domain validation for titles, dates, lengths, and state transitions.
4. Build list overview, task rows, create/edit flow, complete/reopen, reorder, archive, and delete with confirmation where appropriate.
5. Implement keyboard-first entry and quick-add. Ensure focus returns to a sensible location after saving or completing a task.
6. Implement due/overdue visual states without relying on colour alone.
7. Add search/filter only after core CRUD and sorting are stable.
8. Test database restart persistence, migrations, duplicate submissions, invalid input, and concurrent updates.

Deliverables

- Persistent task and list management.
- CRUD and validation layer.
- Task interaction components.
- Automated persistence and domain tests.

Verification

- Do not add reminders/notifications to tasks until the reminder scheduler phase.
- Do not build a complex project-management system beyond the agreed PRD scope.

EXIT GATE — Users can create, edit, complete, reopen, sort, archive, and retrieve tasks after restart without data loss or duplicate records.

Phase 09 — Implement reminders, timers, alarms and scheduling engine

Objective: Build a dependable local scheduling service shared by reminders, timers, and alarms rather than three unrelated timer implementations.

Exact execution sequence

1. Define scheduling semantics: one-shot reminders, repeating reminders, countdown timers, alarms, snooze, dismiss, missed events, and time-zone behaviour.
2. Implement a Rust scheduler service with durable scheduled items, monotonic countdown tracking where appropriate, wall-clock reconciliation, and a clean event interface to the frontend.
3. Persist schedule state before acknowledging creation. Ensure process restart can reconstruct pending events.
4. Implement timer controls: start, pause, resume, reset, cancel, completion, and multiple concurrent timers if allowed by the PRD.
5. Implement reminders: title, optional details, scheduled time, repeat rule, enabled state, completion/dismissal, and edit flow.
6. Implement alarms: time, repeat days, label, sound/vibration policy where applicable, snooze duration, and dismiss.
7. Handle Windows sleep/resume, clock changes, time-zone changes, daylight-saving changes, and missed events. Define whether a missed event fires on resume or is marked missed.
8. Implement notification delivery as a separate adapter so scheduler logic can be tested without OS notifications.

9 Add safeguards against duplicate firing, scheduler race conditions, and duplicate notifications after restart.

10 Implement clear states for permission denied, notification disabled, scheduler unavailable, and missed event.

Deliverables

- Shared scheduling engine.
- Durable timers, reminders and alarms.
- Repeat/snooze/missed-event rules.
- Scheduler integration and restart tests.

Verification

- Do not rely on a JavaScript setInterval as the authoritative scheduler.
- Do not assume a background webview will stay alive indefinitely.
- Do not promise alarm-clock reliability equivalent to a dedicated hardware alarm if Windows sleep/power policies prevent it.

EXIT GATE — Scheduled items fire once according to defined semantics, survive app restart, behave predictably after sleep/resume and clock changes, and cannot duplicate-fire due to races.

Phase 10 — Implement the temporary file staging/dropzone

Objective: Deliver MILO's distinctive file-holding workflow with safe, explicit, non-destructive staging and dependable drag-out to other applications.

Exact execution sequence

1 Write a technical spike before the production UI: determine how a Windows-native file drag source can be initiated from the MILO window and how it will expose file paths using the correct shell data object/OLE drag-drop mechanism.

2 Prototype drag-out to at least File Explorer, a browser upload control, and a representative desktop application. Validate copy semantics, cursor feedback, cancellation, and target compatibility.

3 Define staging model: original absolute path, display name, size, type, added time, optional source metadata, and a stable staging ID. Never store file contents by default.

4 Define whether staged entries persist across restarts and what happens if the source file is moved, renamed, deleted, or becomes inaccessible.

5 Implement file ingestion from OS drag-in and approved file picker. Validate paths and file metadata without reading file contents unnecessarily.

6 Implement a temporary holding area UI with file cards/rows, type icons, names, sizes, timestamps, and missing/unavailable state.

7 Implement explicit remove-from-staging. Removing an item must only remove its staging reference; it must not delete the original file.

8 Implement native drag-out from MILO to external targets. Prefer copy semantics and clearly indicate the requested operation. Do not silently move or delete the source.

9 Support multi-file drag where the native implementation allows it. Test long paths, Unicode paths, network paths, cloud placeholder files, large files, and files that disappear during a drag.

10 Implement fallback actions: Open containing folder, Copy path, Open file, and retry/reselect if a drag target cannot accept the file.

11 Add security controls: reject unsupported special paths, avoid following unsafe links unexpectedly, validate access, and never execute a staged file.

12 Run a dedicated compatibility matrix across browsers, Office-style apps, chat apps, Explorer, and common upload targets.

Deliverables

- Validated Windows drag-out proof of concept.
- Persistent or session-based staging model per PRD.
- Non-destructive staging UI and actions.
- Native OLE/Windows drag source adapter.
- External target compatibility report.

Verification

- Do not simulate drag-out with a browser-only HTML drag event if it cannot deliver real file paths to Windows applications.
- Do not move files into an app-managed temporary directory as a shortcut unless the product requirements explicitly approve a copy workflow.
- Do not delete source files after a successful drop.
- Do not start the polished file dropzone UI before the native drag-out spike succeeds.

EXIT GATE — Dragging staged files out of MILO works on the supported target matrix; cancelled drags leave originals untouched; remove-from-staging never deletes originals; and users can always understand whether an item is staged, missing, or inaccessible.

Phase 11 — Implement media controls and now-playing metadata

Objective: Connect MILO to Windows media sessions with a clear provider-independent model and graceful fallbacks.

Exact execution sequence

1 Research and verify the Windows media session API supported by the target Windows versions. Build a Rust spike that enumerates active sessions and reads metadata.

2 Implement a media service abstraction for current session, playback status, title, artist, album, artwork reference, timeline/position, and supported transport commands.

3 Implement play/pause, next, previous, and seek only when the active session reports support. Do not show enabled controls for unsupported actions.

4 Handle multiple media sessions and define deterministic selection: most recently active, currently playing, user-selected, or another PRD-approved policy.

5 Subscribe to session and metadata changes. Ensure service reconnects after app sleep/resume and player restart.

6 Build the Music section and compact now-playing presentation with artwork fallback, metadata truncation, playback status, progress and control states.

7 Test supported players, browser media sessions, no active player, stale sessions, session switching, protected/unknown metadata, and unavailable artwork.

8 Keep media control independent from lyrics so playback remains useful even when lyrics cannot be retrieved.

Deliverables

- Windows media session adapter.
- Provider-neutral media domain model.
- Now-playing section and controls.
- Session switching/recovery tests.

Verification

- Do not hardcode one music provider.
- Do not scrape private player interfaces or inject into other applications.
- Do not make media controls depend on lyrics availability.

EXIT GATE — At least the agreed supported media players can be detected and controlled accurately; unavailable capabilities are disabled gracefully; and stale session data is not presented as current.

Phase 12 — Implement lyrics capability with source, rights and fallback rules

Objective: Add lyrics only through a permitted and technically supportable source; treat lyrics as an optional enhancement, not a dependency of Music.

Exact execution sequence

1. Define the lyrics source strategy and obtain legal/licensing review for the chosen provider, API, caching, display, and attribution requirements.
2. Define matching logic using title, artist, album, duration, and provider IDs where available. Avoid presenting lyrics for an ambiguous match.
3. Implement a lyrics provider interface with explicit states: unavailable, searching, matched, timed, plain text, no match, provider error, and rate limited.
4. Implement provider authentication only if required; store secrets securely and never embed private API keys in the frontend bundle or public source.
5. Implement attribution, source link, caching TTL, and deletion/retention behaviour according to provider terms.
6. Implement synchronised line timing only where the source supplies licensed timing data. Do not fabricate karaoke timing from plain lyrics.
7. Build lyrics UI with loading, no match, unavailable, and provider error states. Keep typography readable and respect reduced motion.
8. Test track changes, repeated tracks, title/artist mismatches, explicit/clean versions, instrumental tracks, network loss, and provider rate limits.

Deliverables

- Provider abstraction and legal/technical decision record.
- Correct matching and attribution.
- Lyrics presentation and fallback states.
- Provider failure and cache tests.

Verification

- Do not scrape lyric websites or bypass paywalls/access controls.
- Do not ship an unlicensed lyrics corpus.

- Do not fabricate lyrics or line-by-line timing.

EXIT GATE — Lyrics appear only when a valid, permitted match is available; attribution and provider rules are respected; otherwise the Music section remains fully functional without lyrics.

Phase 13 — Implement the MILO avatar and companion behaviour

Objective: Introduce the avatar as a carefully controlled personality layer that enhances the experience without becoming intrusive.

Exact execution sequence

1. Prepare the approved avatar assets and animation inventory. Confirm formats, licensing, transparency, frame rates, dimensions, and file size.
2. Define avatar states: idle, greeting, listening/attention if applicable, success, reminder/notification, error/concern, and hidden. Keep the state set small and purposeful.
3. Implement an avatar controller that receives semantic events rather than having each screen directly manipulate animation files.
4. Ensure avatar appears only in the expanded deck as specified, and does not obstruct controls or content.
5. Implement animation playback with reduced-motion alternatives and a static fallback.
6. Add throttling/cooldowns so repeated events do not trigger noisy animations.
7. Test avatar visibility during navigation, resize, collapse, notification, sleep/resume, and reduced-motion mode.
8. Keep all companion copy and behaviours respectful, optional where appropriate, and non-anthropomorphic about system certainty.

Deliverables

- Optimised avatar assets.
- Semantic avatar state controller.
- Expanded-only placement and animation states.
- Reduced-motion and fallback behaviour.

Verification

- Do not build a complex chatbot/personality engine.
- Do not animate continuously in the background.
- Do not let avatar assets materially degrade launch time, memory, or rendering.

EXIT GATE — The avatar appears in the correct contexts, responds to a small set of meaningful events, never blocks essential tasks, and has a complete reduced-motion/static fallback.

Phase 14 — Implement notifications, quick actions and cross-feature orchestration

Objective: Connect the product modules into one coherent desktop assistant experience.

Exact execution sequence

1. Implement the Windows notification adapter and user permission/settings flow. Confirm the actual behaviour of native notifications on the supported Windows versions.
2. Connect scheduler events to reminders, timers and alarms. Add notification action buttons only where Windows supports the desired interaction.
3. Define notification deduplication, grouping, quiet hours, priority, expiry, and click-through behaviour.
4. Implement quick actions from the island: create task, start timer, add reminder, open files, reveal music, and other PRD-approved actions.
5. Implement global shortcut registration only for agreed shortcuts. Handle conflicts, unregister on shutdown, and provide a settings surface.
6. Implement cross-feature navigation: notification click opens the relevant section and record; timer completion can reveal the deck if permitted by user settings.
7. Ensure system events and background notifications do not steal focus unexpectedly.
8. Add event tracing for diagnosing why a reminder fired, why a notification was suppressed, or why an action failed.

Deliverables

- Native notification integration.
- Quick-action system.
- Global shortcut support.
- Cross-feature event orchestration and diagnostics.

Verification

- Do not request notification permission before explaining why it is needed.
- Do not make notifications the only way to discover a scheduled event.
- Do not introduce unapproved global shortcuts.

EXIT GATE — Notifications are delivered according to settings, deduplicated, actionable where supported, and never cause unexpected focus theft; quick actions navigate to the correct feature.

Phase 15 — Complete settings, onboarding, accessibility and resilience

Objective: Make the application understandable, configurable, recoverable, and comfortable for daily use.

Exact execution sequence

1. Implement first-run onboarding with a concise explanation of MILO, how to reveal it, core privacy behaviour, and optional permission requests at the point of need.
2. Implement settings for startup behaviour, appearance, motion, notifications, sound, hotkeys, default monitor, island behaviour, and data/diagnostics as defined in the PRD.
3. Use progressive permission requests. Ask only when a user activates the feature that needs the permission; explain what is requested and why.
4. Implement data export and safe reset flows. Clearly distinguish resetting preferences from deleting user-created tasks/reminders and other data.
5. Implement settings persistence and migration. Validate settings and recover from corrupt or missing preferences.
6. Complete keyboard navigation, screen-reader semantics, high contrast considerations, text scaling, focus visibility, reduced motion, and accessible error messaging.

7 Test Windows display scaling, narrow layouts, large text, long task names, long file names, localisation fallback, and content overflow.

8 Implement offline and partial-connectivity behaviour. Core local features should remain useful without a network connection.

9 Implement crash recovery, corrupted database handling, graceful service restarts, and user-readable diagnostics.

10 Review all user-facing copy for clarity, consent, destructive-action warnings, and consistency with MILO's personality.

Deliverables

- Onboarding and settings.
- Permission and privacy UX.
- Accessible interaction coverage.
- Export/reset and recovery flows.
- Offline/partial-failure behaviour.

Verification

- Do not add a mandatory account or cloud sync.
- Do not make diagnostics silently transmit user data.
- Do not put destructive reset behind ambiguous labels.

EXIT GATE — A first-time user can understand and configure MILO; core local features work offline; permissions are requested contextually; and common failure modes are recoverable without data loss.

Phase 16 — Performance, power, memory and visual polish

Objective: Optimise MILO as a lightweight always-available desktop companion rather than a resource-heavy web app.

Exact execution sequence

1 Capture a performance baseline for cold launch, warm reveal, collapse, section switch, system refresh, database operations, and media changes.

2 Profile frontend render counts, animation frame times, WebView2 memory, Rust process CPU, native polling, event subscriptions, and database access.

3 Set explicit budgets for idle CPU, memory, startup time, interaction latency, and animation smoothness. Derive numeric thresholds from the TRD and record test hardware.

4 Replace unnecessary polling with native events or adaptive polling. Back off when hidden, idle, or on battery where appropriate.

5 Lazy-load nonessential sections and avatar assets. Avoid oversized image assets and unbounded caches.

6 Audit event listeners, timers, file handles, native subscriptions, and database connections for leaks.

7 Verify the collapsed state has minimal work and that hidden/idle states do not animate or poll unnecessarily.

8 Polish transitions, shadows, blur, gradients, hover, focus, pressed, disabled, empty and error states against the design debrief.

9 Run long-duration soak tests, repeated open/close cycles, multi-hour timers, repeated media switching, and sleep/resume cycles.

Deliverables

- Performance profile and budget report.
- Reduced idle work and memory leaks.
- Polished visual and motion states.
- Soak-test results.

Verification

- Do not optimise based only on developer-machine impressions.
- Do not remove useful feedback or accessibility to improve a benchmark.
- Do not add perpetual animation or polling to make the app feel alive.

EXIT GATE — Performance meets the TRD budgets on documented hardware; the idle/collapsed app is lightweight; no sustained resource leak appears during soak tests; and polish has not reduced accessibility or reliability.

Phase 17 — Comprehensive QA, compatibility and security review

Objective: Validate the complete product as a real Windows application, not just a developer build.

Exact execution sequence

1. Build a requirement-to-test traceability matrix. Every must-have requirement must map to at least one automated or documented manual test.
2. Run unit tests for domain logic, scheduler rules, settings validation, file staging, media matching, and error mapping.
3. Run integration tests for IPC, database migrations, native service adapters, notification delivery, and lifecycle recovery.
4. Run end-to-end tests for first launch, reveal/collapse, navigation, tasks, reminders, timers, file staging/drag-out, media, settings, and quit/relaunch.
5. Run the full Windows compatibility matrix: Windows versions in scope, x64/ARM64 if supported, DPI, multiple monitors, laptop/desktop, battery/no battery, Bluetooth/Wi-Fi availability, and WebView2 states.
6. Run adversarial file tests: Unicode, long paths, network locations, cloud placeholders, inaccessible paths, missing files, large batches, cancelled drag, and target rejection.
7. Run scheduler tests around daylight-saving transitions, time-zone changes, sleep/resume, missed alarms, clock correction, and duplicate events.
8. Perform security review: Tauri capability allowlist, command input validation, path handling, shell usage, CSP, dependency audit, secret storage, updater signature verification, and log redaction.
9. Perform privacy review: inventory data stored, data retained, data transmitted, permissions, diagnostics, lyrics provider, and third-party dependencies. Confirm no hidden analytics or account requirement.
10. Conduct usability testing with fresh users. Observe discoverability of the collapsed island, understanding of pinning, file staging semantics, and recovery from errors.
11. Fix release-blocking defects first. Classify remaining issues as blocker, critical, major, minor, or cosmetic. No blocker/critical issue may ship.
12. Repeat regression suite after fixes and archive evidence.

Deliverables

- QA matrix and traceability report.
- Security and privacy review.

- Compatibility and usability reports.
- Release-blocking defect list cleared.
- Regression evidence archive.

Verification

- Do not treat a successful compile as QA.
- Do not waive a file safety, data-loss, scheduler, permission, or installer defect as cosmetic.
- Do not claim support for an OS/device configuration that has not been tested.

EXIT GATE — All must-have requirements are verified; no blocker or critical defect remains; security/privacy reviews are signed off; and the supported Windows matrix has been exercised.

Phase 18 — Production packaging, installer, signing and update pipeline

Objective: Turn the validated application into a professional Windows product that users can install, launch, update, repair, and uninstall.

Exact execution sequence

1. Confirm product metadata: publisher identity, app name, bundle ID, version, icon set, legal notices, support URL, privacy policy, and release notes format.
2. Choose the Windows installer format(s) based on user distribution goals. Validate install scope, per-user/per-machine behaviour, upgrade semantics, and uninstall behaviour.
3. Configure production Tauri bundling and verify all runtime assets, database migrations, icons, WebView2 assumptions, and native dependencies are included.
4. Set up code signing with a real certificate when available. Store signing material in secure CI secrets; never commit private keys or certificates.
5. Configure updater signing and public-key verification. Keep release endpoints and channels explicit. Test a signed update from an older build.
6. Implement release automation: version bump, changelog, clean build, test suite, signing, artifact generation, checksums, SBOM/dependency inventory if required, and release publication.
7. Test fresh install, first launch, upgrade from previous version, downgrade policy, repair/reinstall, uninstall, and clean reinstall.
8. Verify user data preservation across upgrades and uninstall. Clearly define whether uninstall preserves user data and provide an explicit data-removal option if appropriate.
9. Test installer behaviour under standard user accounts, restricted corporate devices, antivirus scanning, SmartScreen reputation, and offline installation where supported.
10. Create a rollback/disable-update plan for a faulty release. Do not force-install unverified updates.

Deliverables

- Reproducible Windows installer.
- Signing and updater configuration.
- Release CI pipeline.
- Upgrade/uninstall test evidence.
- Rollback and release operations notes.

Verification

- Do not publish unsigned production binaries as if they were a trusted final release.
- Do not place signing secrets in source control or frontend code.
- Do not enable automatic updates until the signed update path has been tested end-to-end.

EXIT GATE — A clean Windows machine can install and launch MILO; a previous version can update safely; signatures verify; user data survives intended upgrade paths; and uninstall behaviour is clear and tested.

Phase 19 — Pilot release, feedback, final acceptance and handover

Objective: Release to a controlled group, resolve real-world issues, and formally close the initial implementation.

Exact execution sequence

1. Prepare a pilot build with version number, release notes, known limitations, support channel, and diagnostic instructions.
2. Distribute to a small pilot group on varied Windows hardware. Include laptops, desktops, multiple-monitor users, mixed-DPI setups, and users of different media players.
3. Collect opt-in feedback and diagnostics with clear consent. Avoid collecting file names, file contents, personal task text, or other sensitive data by default.
4. Track crashes, startup failures, missed/duplicate reminders, window positioning issues, drag-out failures, media session errors, and resource use.
5. Prioritise issues by severity and frequency. Fix data-loss, security, privacy, scheduler, drag/drop, and launch defects before visual refinements.
6. Run the complete regression suite on the pilot candidate. Rebuild and retest after every release-blocking fix.
7. Complete final acceptance against the PRD and TRD. Product owner signs off each requirement group.
8. Prepare end-user quick-start guide, troubleshooting guide, privacy summary, release notes, and support/diagnostic instructions.
9. Prepare engineering handover: architecture overview, module map, ADRs, build commands, test commands, release procedure, secrets management, updater operations, known limitations, and recovery instructions.
10. Tag the release in source control, archive build artifacts and test evidence, and create the next-version backlog without silently expanding the initial release scope.

Deliverables

- Pilot candidate and feedback report.
- Final acceptance sign-off.
- User documentation.
- Engineering handover package.
- Tagged stable release and archived evidence.

Verification

- Do not call the product finished because the feature list is implemented.
- Do not ship pilot-only debug tools or test credentials.
- Do not silently expand scope during final stabilisation.

EXIT GATE — MILO is installable by the intended users, all acceptance criteria are signed off, pilot blockers are resolved, documentation is complete, and another engineer can build, test, release, and support it from the handover package.

Cross-cutting engineering workstreams

A. Architecture boundaries

Layer	Owns	Must not own
React UI	Presentation, local interaction state, accessibility, navigation.	Direct Windows API calls, filesystem authority, scheduling truth.
Typed IPC client	Typed command/event access, request validation at UI boundary.	Business rules or arbitrary command execution.
Rust domain services	Business rules, validation, orchestration, durable operations.	Visual layout and presentation decisions.
Windows adapters	OS-specific APIs, capability detection, native event subscriptions.	Product copy or React state.
Persistence	Migrations, transactions, durable user data.	Direct UI rendering or unvalidated arbitrary SQL from components.
External providers	Lyrics or other optional integrations, attribution, rate limits.	Core local feature availability.

B. Feature implementation pattern

1. Write a short feature design note: user outcome, requirements, states, data, permissions, native dependencies, and failure modes.
2. Prove any Windows-only or third-party dependency with a minimal spike.
3. Define domain types, validation, persistence, and IPC contract.
4. Implement backend/domain logic and unit tests first where business rules are involved.
5. Implement the frontend interaction using real typed contracts.
6. Add loading, empty, unsupported, permission-denied, error, and recovery states.
7. Add integration and end-to-end tests.
8. Run Windows manual verification and capture evidence.
9. Update docs and mark the feature complete only when the exit gate passes.

C. Error handling contract

All native operations must return structured errors with a stable code, a safe user-facing message, and optional diagnostic context. The frontend should not parse arbitrary OS error strings to decide behaviour. Errors must distinguish a device that does not support a capability from a supported device whose operation failed.

Error class	Expected UI response
Unsupported	Show the capability as unavailable; explain briefly; do not present a broken control.
Permission denied	Explain the required permission and offer a route to enable it if possible.
Temporarily unavailable	Show a retry/reconnect state and retain safe user input.
Validation	Keep the form open and identify the invalid field.
Conflict/stale state	Refresh the current state and explain that it changed elsewhere.

Error class	Expected UI response
Persistence failure	Do not claim success; preserve entered data and provide a recovery path.
Internal error	Show a calm message and offer diagnostics without exposing internals.

D. Privacy and data handling rules

- Core MILO features must work without an account.
- Tasks, reminders, settings, staged file references, and local preferences remain on-device unless the user explicitly chooses otherwise.
- A staged file is a reference to an existing file, not a copy of its contents. The original is never altered by staging or removing the staging reference.
- No file contents, personal task text, or full file paths should be sent to a remote service by default.
- Lyrics provider use must comply with provider terms and must be isolated from core music controls.
- Diagnostics are local by default. Any sharing must be explicit, reviewable, and redacted.
- No hidden telemetry, advertising, account creation, or cloud sync should be introduced without a product decision and updated privacy documentation.

Repository structure recommendation

Adapt this layout to the repository that exists; do not force a restructure if an equivalent, clean structure is already present.

```
milo/ docs/ architecture/ adr/ implementation/ qa/ release/ src/ app/ components/ primitives/ shell/
feedback/ features/ overview/ system/ tasks/ reminders/ timers/ alarms/ files/ music/ lyrics/
avatar/ settings/ domain/ state/ services/ ipc/ events/ persistence/ styles/ assets/ test/
src-tauri/ src/ commands/ domain/ services/ platform/ windows/ persistence/ scheduler/ events/
logging/ capabilities/ icons/ tests/ migrations/ scripts/ .github/workflows/ package.json Cargo.toml
README.md
```

Recommended milestone and review cadence

Milestone	Phases closed	Review question
M0 — Environment proven	0–1	Can a new engineer reproduce the build and understand the codebase?
M1 — Native shell proven	2, 5	Does the Windows island behave correctly as a desktop window?
M2 — UI foundation ready	3–6	Is the design system and shell stable enough to host real features?
M3 — Core utility ready	7–9	Do system controls and productivity data work reliably?
M4 — Signature workflows ready	10–13	Do file staging, media, lyrics and avatar work with honest fallbacks?
M5 — Daily-use ready	14–16	Are notifications, settings, accessibility and performance ready?
M6 — Release candidate	17–18	Has the complete Windows app passed QA and packaging tests?
M7 — Stable release	19	Has the pilot passed and is the handover complete?

At each milestone review, Codex must provide: (1) what was built, (2) the exact files/modules changed, (3) test commands and results, (4) a short manual verification record, (5) open defects and risks, and (6) confirmation that the next phase's dependencies are satisfied.

Suggested acceptance scorecard

Area	Release acceptance
Launch and lifecycle	No duplicate instances; predictable tray, startup, shutdown, restart and recovery.
Floating shell	Correct top-edge positioning, focus, pinning, collapse/expand, multi-monitor and DPI behaviour.
System deck	Real state and supported controls; honest unsupported states; no fake toggles.
Productivity	Tasks, reminders, timers and alarms persist and follow documented scheduling semantics.
File staging	Real external drag-out; original files preserved; cancel/remove are non-destructive.
Music and lyrics	Media sessions accurately represented; lyrics optional, correctly matched and legally sourced.
Avatar	Expanded-only, event-driven, non-obstructive, reduced-motion safe.
Privacy and security	Least privilege, local-first, safe file paths, no hidden data transfer, signed updates.
Accessibility	Keyboard, focus, readable states, screen-reader labels, reduced motion and scaling.
Performance	Meets documented budgets; low idle resource use; no leaks in soak tests.
Distribution	Installer, signatures, upgrade, uninstall and rollback tested.
Operations	Documentation, diagnostics, release process and engineering handover complete.

Critical technical spikes — perform early

Spike	When	Proof required
Floating transparent top-edge window	Phase 2	Native window remains positioned and interactive through DPI/monitor changes.
Tauri capability/security model	Phase 1–4	Frontend can invoke only intended commands with least privilege.
Windows system controls	Phase 7	Volume/brightness/power/network capabilities are honestly detected and controlled.
Durable scheduling	Phase 9	Timer/reminder state survives process termination and sleep/resume without duplicate firing.
Native external file drag	Phase 10	Real file paths are exposed to external Windows applications using native drag/drop.
Windows media sessions	Phase 11	Enumerate sessions, read metadata, control supported actions, recover on player changes.
Lyrics rights/source	Phase 12	Permitted source, matching, attribution, caching and failure policy documented.
Signed updater	Phase 18	A signed build can verify and install a signed update from a prior release.

What Codex must never do

- Never implement all screens as static mockups and call the app complete.
- Never replace a failed native capability with a fake UI toggle.
- Never build the file staging experience on an unproven browser-only drag event.
- Never make a staged file operation destructive by default.
- Never use JavaScript timers as the source of truth for alarms or reminders.

- Never scrape protected media or lyrics interfaces, bypass access controls, or ship unlicensed lyrics.
- Never store secrets in the frontend bundle, repository, or plain-text settings.
- Never request broad filesystem, shell, or network permissions without a specific requirement.
- Never swallow native errors or report success before the OS confirms the operation.
- Never allow animation, avatar behaviour, or hover logic to block essential interaction.
- Never skip migration, upgrade, and uninstall testing.
- Never move to the next phase while the current exit gate is failing.

Copy-ready master instruction for Codex

Use this as the standing execution instruction when beginning the implementation. Keep this plan and the companion MILO documents in the repository under docs/ so they remain available to future Codex sessions.

```
You are implementing MILO, a Windows-first floating desktop application. Before writing code, read the MILO PRD, Technical Requirements Document, UI/UX Design Debrief, and Implementation Plan in full. Follow the Implementation Plan phases strictly in order. First inspect the repository and report its current state. Do not overwrite working code or assume the repository is empty. Create a requirement traceability checklist and phase checklist. For each phase: 1. State the objective and dependencies. 2. Identify the smallest coherent implementation slice. 3. Prove risky Windows/native dependencies with a minimal spike before building dependent UI. 4. Implement the backend/domain contract and tests where applicable. 5. Implement the frontend against typed IPC contracts. 6. Include loading, empty, unsupported, permission-denied, error and recovery states. 7. Run formatting, lint, type-check, Rust checks, unit/integration tests, and a production build. 8. Perform required manual Windows verification and record the result. 9. Update docs, ADRs, and the traceability checklist. 10. Report files changed, commands run, results, open risks, and whether the phase exit gate passed. Do not start the next phase until the current phase's exit gate passes. Do not fake functionality. Do not silently move, delete, overwrite, or upload user files. Do not use JavaScript intervals as the authoritative scheduler. Do not scrape protected interfaces or use unlicensed lyrics. Use least privilege and explicit Tauri capabilities. Keep core functionality local-first and account-free. If a required Windows behaviour cannot be proven in the current environment, do not claim it works. Create a minimal reproducible spike, document the limitation, and request a real Windows verification step. Prefer a small, tested, maintainable implementation over a large speculative rewrite. The goal is a complete, reliable, polished, installable Windows application—not a visual prototype.
```

Official technical references to validate during implementation

These links are starting points, not a substitute for checking the exact API, crate, plugin, permission, and Windows-version requirements at the moment of implementation. Codex must verify current documentation and pin compatible versions before adopting an API.

Tauri v2 architecture — Frontend WebView and Rust/native integration model.
https://v2.tauri.app/concept/architecture/

Tauri v2 plugin catalogue — Official plugin capabilities such as filesystem, notifications, global shortcuts, SQL, store, and updater.
https://v2.tauri.app/plugin/

Tauri v2 updater — Signed update artifacts, endpoints, platform targets and Windows install modes.
https://v2.tauri.app/plugin/updater/

Microsoft Learn — Windows drag and drop — Windows drag/drop concepts, data packages, operations and target handling.
https://learn.microsoft.com/en-us/windows/apps/design/input/drag-and-drop

Microsoft Learn — drag/drop data development — Data package construction, drag start/drop completion and file transfer patterns.
https://learn.microsoft.com/en-us/windows/apps/develop/data/drag-and-drop

Implementation note: MILO's native file drag-out and floating window behaviour may require Windows-specific Rust/Win32 work beyond a standard cross-platform UI abstraction. Keep that code isolated behind platform adapters and test it on actual Windows machines.

Final instruction

MILO is finished only when the complete application has passed its phase gates, the Windows installer and update path have been tested, user data is safe, the pilot is accepted, and the engineering handover is complete. A running development window is not the finish line.